

Natural Language Processing

Session 06 – Language Models & Word Embeddings

Prof. Dr. Richard Sieg
SoSe 2026

Schedule

Date	Weekday	CW	Room	Topic
16.04.	Thursday	16	149	Introduction: The World of NLP
23.04.	Thursday	17	149	Text Processing Fundamentals
30.04.	Thursday	18	149	Linguistic Annotations & Pattern Matching
05.05.	Tuesday	19	154	Text Vectorization: From Text to Numbers ⚠️ Substitute for 14.05.
07.05.	Thursday	19	149	Text Classification
28.05.	Thursday	22	149	Language Models, Word Vectors + 🎤 Guest Lecture Ines Montani
01.06.	Monday	23	390	📝 Exams — Master DS Students
02.06.	Tuesday	23	154	BERT: Pre-Training & Fine-Tuning ⚠️ Substitute for 04.06.
11.06.	Thursday	24	149	The BERT Ecosystem
18.06.	Thursday	25	149	Introduction to LLMs
25.06.	Thursday	26	149	Working with LLMs
02.07.	Thursday	27	149	Ethics, Limitations & Exam Preparation
09.07.	Thursday	28	149	📝 Exams (afternoon) — Bachelor DIS Students
10.07.	Friday	28	149	📝 Exams (afternoon) — Bachelor DIS Students

Guest Lecture Ines Montani

Our lecture is until 4.15.

Ines talk starts 4.30 in room 69.

Crucial background lecture today,
no assignment



Co-founder Explosion & creator of spaCy

ines.io

Recap: Sparse Vectors

We've been representing text as bag-of-words vectors, one dimension per vocabulary word, counts as values.

	the	money	road	home	bass	dirt
Lyric A: "money cash money grind"	0	3	0	0	0	0
Lyric B: "down the dirt road"	1	0	1	0	0	1

Result: each document is a sparse, high-dimensional, weighted vector. Vocabulary size = vector dimensionality. Typical: 20,000 — 50,000+.

What's Wrong With Sparse Vectors?

⚠ They're huge and mostly zero.

A 50k-dim vector with maybe 50 non-zero entries per document.

⚠ Each word is independent.

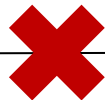
To the model, "car" and "automobile" are as different as "car" and "banana".

⚠ No notion of similarity between words.

The dot product of one-hot vectors for "cat" and "dog" is zero — exactly the same as "cat" and "stapler".

No transfer of knowledge

This is song is *terrible*.



This is song is *horrible*.

Meaning From Context

How would you figure out what an unfamiliar word means?

Ong choy is delicious sauteed with garlic. Ong choy is superb over rice. ...ong choy leaves with salty sauces...

You've never seen "ong choy" before. But you've seen spinach, chard, kale appear in exactly these contexts:

...spinach sauteed with garlic over rice...

...chard stems and leaves are delicious...

...kale and other salty leafy greens...

**So ong choy is probably a leafy green.
You inferred meaning from company kept.**



The Distributional Hypothesis

"You shall know a word by the company it keeps."

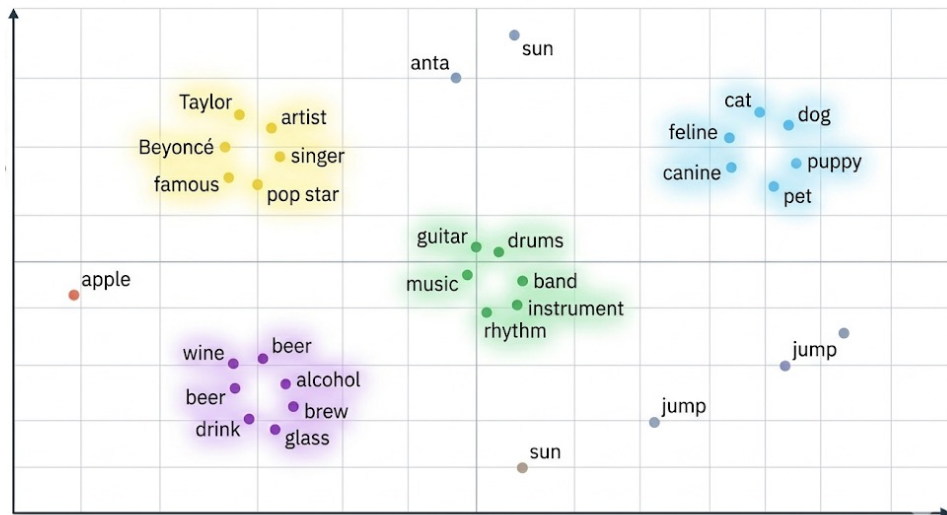
J. R. Firth, 1957



Words that appear in similar contexts tend to have similar meanings.

This is the foundation of every modern word representation: from word2vec to BERT to GPT.

To "capture meaning" computationally = to "capture contexts" statistically.



Agenda

01.

Language Models

02.

Neural Language Models

03.

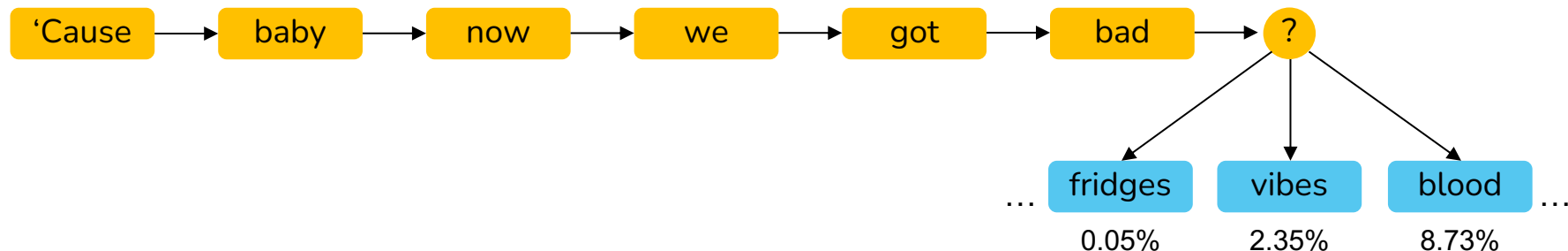
Word2Vec & GloVe

01.

Language Models

What Is a Language Model?

A **language model** predicts the next word given some context (e.g. previous words).



A language model assigns a probability to each candidate word given previous words:

$$P(w_i \mid w_1, \dots, w_{i-1}) = ?$$

Equivalently, it assigns a probability to a whole sentence:

$$P(w_1, w_2, \dots, w_n) = ?$$

Language Models are everywhere

Google

natural lan

- natural language processing
- natural language processing deutsch
- natural language meaning
- natural language processing (nlp)

```
https://jalammar.github.io/illustrated-transformer/  
- Build a large language model (from Scratch) - Sebastian Raschka  
- AI engineering: building applications with foundation models - Chip Huyen  
- Hands-On Large Language Models - Jay Alammar, Maarten Grootendorst  
- [The Illustrated Transformer](https://jalammar.github.io/illustrated-transformer/) Yo
```

Willkommen, Richard

Wie kann ich dir heute helfen?



Sonnet 4.5



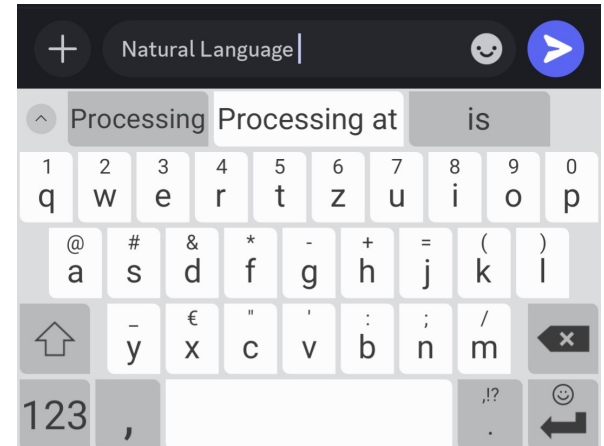
</> Code

Lernen

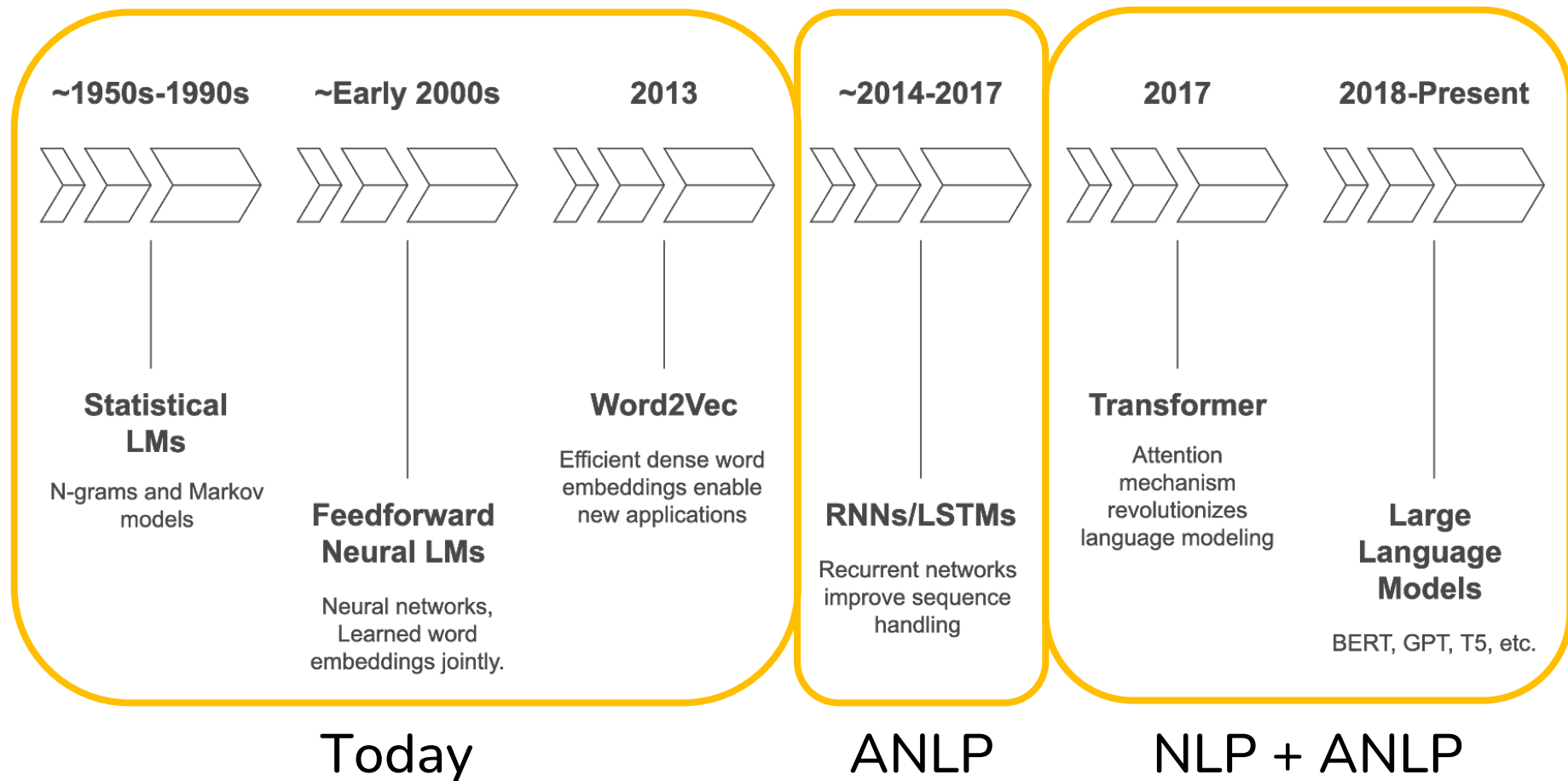
Schreiben

Privates

Aus Calendar



History of Language Models



Counting Sentences

- How do we estimate $P(w_1, w_2, \dots, w_n)$?
- *First idea*: Just count whole sentences in a corpus.

$$P(\text{shake it off}) = \frac{\text{count}(\text{shake it off})}{\text{count}(\text{all sentences})}$$

⚠ Most sentences have never been said before. *Language has infinite possibilities!*

- We could decompose the sentence into smaller pieces using the *chain rule*

$$P(w_1, w_2, \dots, w_n) = \prod_{i=1}^n P(w_i \mid w_1, \dots, w_{i-1})$$

$$P(\text{shake, it, off}) = P(\text{shake}) \cdot P(\text{it} \mid \text{shake}) \cdot P(\text{off} \mid \text{shake, it})$$

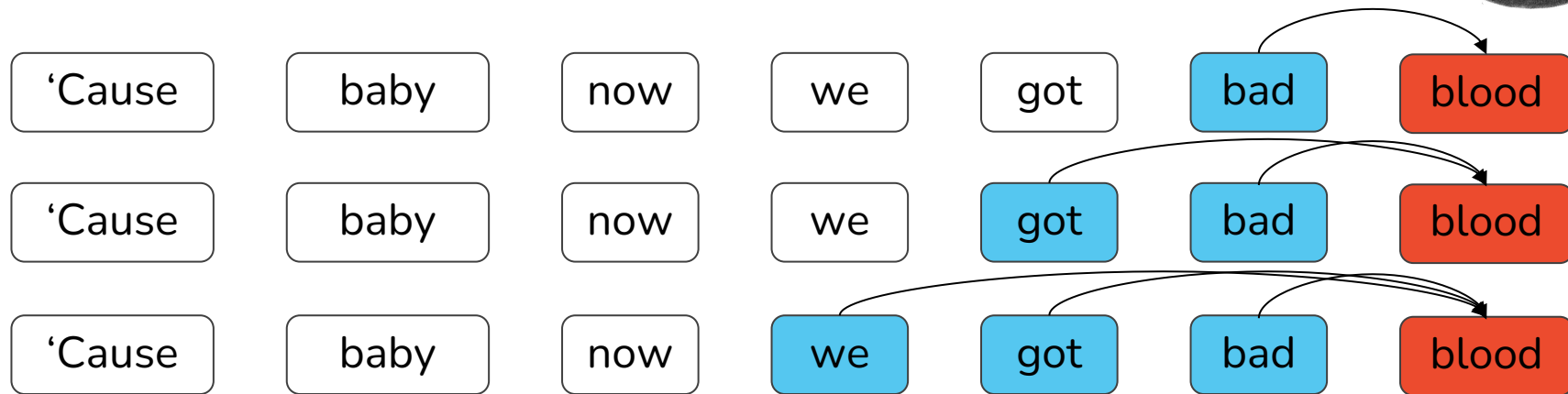
- This is exact, but it does not actually help yet.
Computing $P(\text{blood} \mid \text{'Cause baby now we got bad})$ still requires counting how often that exact 6 word context appears in a corpus.

The Markov Assumption

- **Key idea:** Don't condition on the whole history. Condition on only the last few words.

$$P(w_i | w_1, \dots, w_{i-1}) \approx P(w_i | w_{i-N+1}, \dots, w_{i-1})$$

- This gives us a whole family of possible ***N-gram models***:
 - Unigram (N=1): $P(w_i)$ - no context at all
 - Bigram (N=2): $P(w_i | w_{i-1})$ - one word of context
 - Trigram (N=3): $P(w_i | w_{i-2}, w_{i-1})$ - two words of context



Generating from N-Grams

Use maximum likelihood estimation: count and divide, e.g. for bigram:

$$P(w_i | w_{i-1}) = \frac{\text{count}(w_{i-1}, w_i)}{\text{count}(w_{i-1})}$$

<s> shake it off </s>

<s> shake it shake it </s>

<s> i knew you were trouble </s>

$$P(\text{shake} | \text{<s>}) = \frac{2}{3} \quad P(\text{it} | \text{shake}) = 1.0 \quad P(\text{off} | \text{it}) = \frac{1}{3}$$

N	Sample output
Unigram	you the i love my and night know want them go
Bigram	i love you know i'm only me when we are never go
Trigram	we are never ever getting back to december turn around
4-gram	shake it off shake it off i shake it off shake

↓
Coherence

↓
Memorization

Evaluating a Language Model: Perplexity

- How good is a language model?
- **Perplexity** measures how "surprised" it is by held-out test text:

$$PP(W) = P(w_1 w_2 \dots w_N)^{-\frac{1}{N}}$$

- *Intuition*: a perplexity of 100 means the model is, on average, as confused as if it had to choose uniformly among 100 words at each step.
- N-gram models on Wall Street Journal (38M training words, 1.5M test words):

Model	Perplexity
Unigram	962
Bigram	170
Trigram	109

Modern neural LMs reach perplexities in the 20–60 range on small benchmarks like Penn Treebank, and below 10 on large-corpus benchmarks with subword tokenization.

Limits of N-gram models

N-gram models are simple, fast, and capture surprisingly much. But they have strong limitations.

1. *They can't handle long-distance dependencies:*

The songs that I heard on the radio on the way home yesterday were amazing.

To predict "were", you need to know about "songs", 11 words back. No realistic N-gram window can reach that far.

2. *They can't generalize across similar contexts:*

$P(w \mid \text{"I am a good"})$ and $P(w \mid \text{"I am a great"})$ are estimated completely independently. Even though "good" and "great" mean almost the same thing, the model has no way to share information between them.

3. *Computationally infeasible:*

An N-gram model has up to $|V|^N$ possible N-grams to track. For a 50,000-word vocabulary, there are 3×10^{23} N-grams to track. Almost all of those n-grams appear zero times in any realistic corpus, so a bigger N mostly buys you more zeros.

Example in marimo

A Neural Probabilistic Language Model

Yoshua Bengio

Réjean Ducharme

Pascal Vincent

Christian Jauvin

Département d'Informatique et Recherche Opérationnelle

Centre de Recherche Mathématiques

Université de Montréal, Montréal, Québec, Canada

BENGIOY@IRO.UMONTREAL.CA

DUCHARME@IRO.UMONTREAL.CA

VINCENTP@IRO.UMONTREAL.CA

JAUVINC@IRO.UMONTREAL.CA

02.

Neural Language Models

Bengio's Big Idea (2003)



- Train a neural network on the language modeling task (predict the next word), and let it learn its own word representations as part of doing the job.
- The vectors are not given to the model. The model learns them from data, by adjusting them to make next-word prediction more accurate.
- Synonyms naturally drift close together because they occur in similar contexts.

Counting (N-gram)	Predicting (Neural LM)
Each word is a discrete symbol	Each word is a vector in $\mathbb{R}^d$
good and great are independent	good and great can be close in the vector space
Estimate $P(w \mid \text{context})$ by counting	Estimate $P(w \mid \text{context})$ by a neural net
New context = no data	Similar context = similar prediction

The Architecture

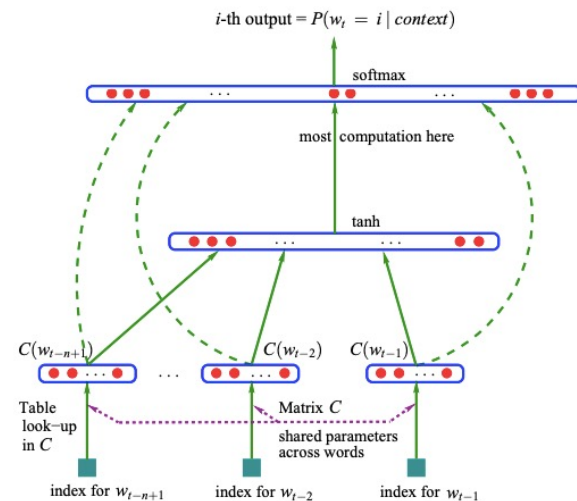
A feedforward neural network that predicts the next word from the previous $n-1$ words.

Forward pass for predicting word w_t :

1. Look up each context word's vector from a shared embedding matrix $C \in \mathbb{R}^{|V| \times d}$.
2. Concatenate the three vectors into a single input $x \in \mathbb{R}^{3d}$.
3. Hidden layer with a non-linearity: $h = \tanh(Wx + b)$.
4. Output layer projects to vocabulary size, then softmax turns scores into probabilities:

$$P(w_t \mid w_{t-3}, w_{t-2}, w_{t-1}) = \text{softmax}(U \cdot \tanh(Wx + b))$$

The whole network is trained by backpropagation to maximize the probability of the actual next word in training data.



The vectors are the byproduct. We trained for language modeling; we keep the embeddings C . Every embedding-based model since (word2vec, GloVe, BERT) is built on this trick.



Why It Was Revolutionary

- First model to learn distributed word representations as part of training a real task.
- Generalization for free: similar words automatically share predictions through their similar vectors.
- Outperformed the best smoothed n-gram models of the day on perplexity benchmarks.
- Established the template for everything that came after: word2vec, GloVe, BERT, GPT. All of them learn vectors as a side effect of a prediction task.



What It Couldn't Do

- Just like n-grams, it only looks at the last few words.
- Parameters scale with window size. Doubling the window doubles the input layer. Going from 3 to 30 words is expensive.
- Slow softmax over the full vocabulary: Computing $P(w_t | \text{context})$ for every word in a 50k vocabulary, for every training example, is the bottleneck. Training Bengio's original model took weeks.

03.

Word2Vec & GloVe

word2vec (2013)

- Bengio gave us word vectors at the cost of training a full language model.
- Mikolov et al. (2013, Google) asked:

*If we only care about the embeddings,
do we really need to train a full language model?*

- Drop the expensive next-word softmax
- Replace it with a much simpler task
- Throw away the model afterwards
- Keep only the embeddings



Self-Supervised Binary Classification

- The simpler task: given a target word and a candidate context word, is this pair real or random noise? That is just binary classification. No softmax over 50,000 words.
- Train a tiny classifier on billions of such pairs. The classifier itself is useless. The vectors it learns are the prize.
- No labels needed. Running text is the training data.

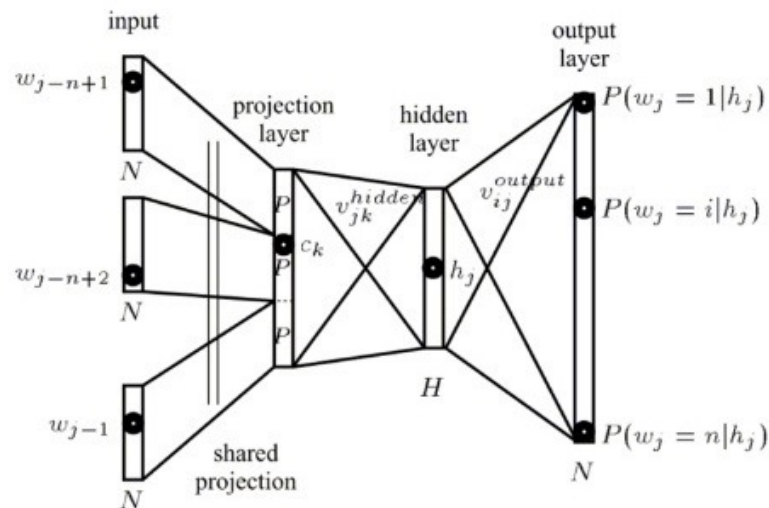
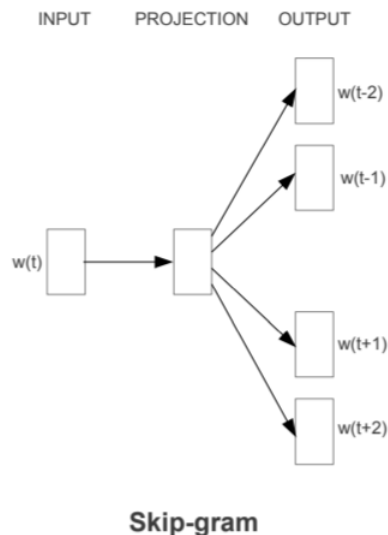
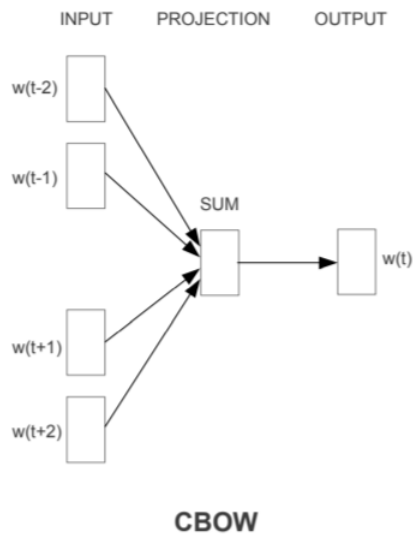
I knew you were trouble when you walked in

- Slide the window across the corpus
- Billions of training pairs, no annotation
- The structure of language is the supervision

Pair	Label
(trouble, knew)	✓ real
(trouble, you)	✓ real
(trouble, aardvark)	✗ noise
(trouble, Tolstoy)	✗ noise

Two Architectures: Skip-gram vs CBOW

- Two flavors. They differ in direction of prediction.
- Skip-gram: center $\rightarrow$ context:
trouble $\rightarrow$ **knew, you, were, when**
- CBOW (Continuous Bag-of-Words): context $\rightarrow$ center
knew, you, were, when $\rightarrow$ **trouble**

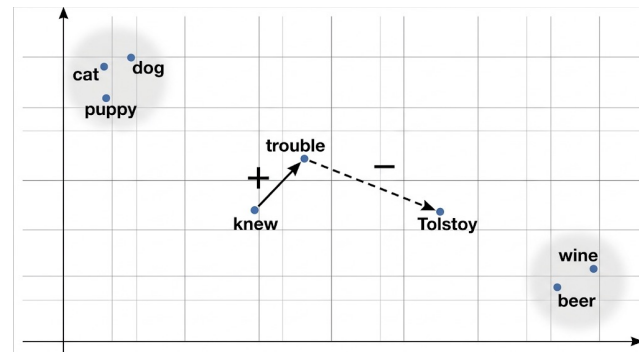


How Learning Works

- For each training pair, adjust the two vectors in opposite directions:
 - Positive pair (trouble, knew) → move vectors closer
 - Negative pair (trouble, Tolstoy) → push vectors apart
- The SGNS (Skip-Gram Negative Sampling) loss function with k negative samples:

$$L = \underbrace{-\log \sigma(\overrightarrow{c_{\text{pos}}} \cdot \vec{w})}_{\text{pull positives closer}} - \underbrace{\sum_{i=1}^k \log \sigma(-\overrightarrow{c_{\text{neg}_i}} \cdot \vec{w})}_{\text{push negatives apart}}$$

- Co-occurring words converge
- Unrelated words drift apart
- The space self-organizes around meaning



Hyperparameters

- Window size. Controls what kind of similarity:
 - Small (± 2) $\rightarrow$ functional similarity (Hogwarts $\approx$ Sunnydale: both are fictional schools)
 - Large (± 10) $\rightarrow$ topical similarity (Hogwarts $\approx$ Dumbledore: both are Harry Potter)
- Embedding dimension. Typically 100 to 300. More dimensions = more nuance, but more memory and overfitting risk.
- Number of negative samples. 5 to 20 for small corpora, 2 to 5 for huge ones.
- Negative sample distribution: $p(w) \propto \text{count}(w)^{0.75}$ (boost rare words)
- FastText (Mikolov, 2015, Facebook): same pipeline as skip-gram, but each word is also a bag of character n-grams (**trouble** $\rightarrow$ <**tr**, **tro**, **rou**, **oub**, **ubl**, **ble**, **le**>).
Handles unknown words, typos, morphology.
Especially useful for German compounds.

GloVe (2014)

- Word2vec learns from local context (one window at a time).
- GloVe (Pennington, Socher, Manning): use global corpus statistics directly.

1. Build the full word-word co-occurrence matrix X from the corpus

2. Learn vectors v_i, u_j such that $v_i \cdot u_j \approx \log X_{ij}$

3. Weight rare and frequent co-occurrences so neither dominates

Loss function:

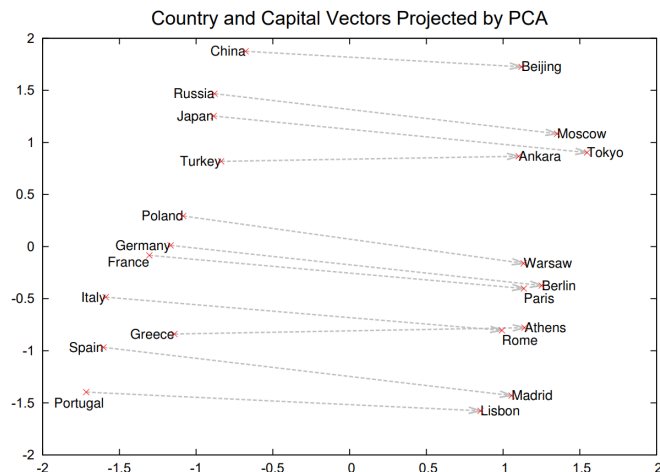
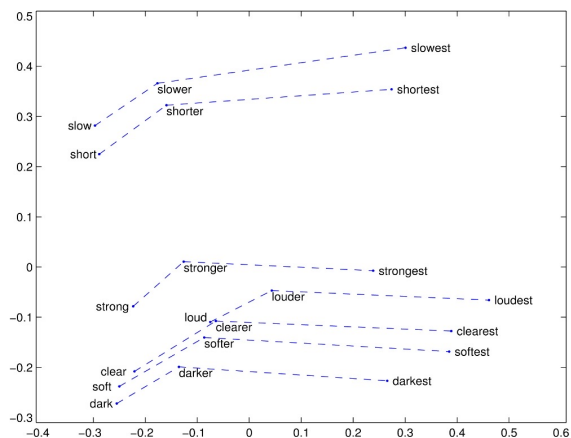
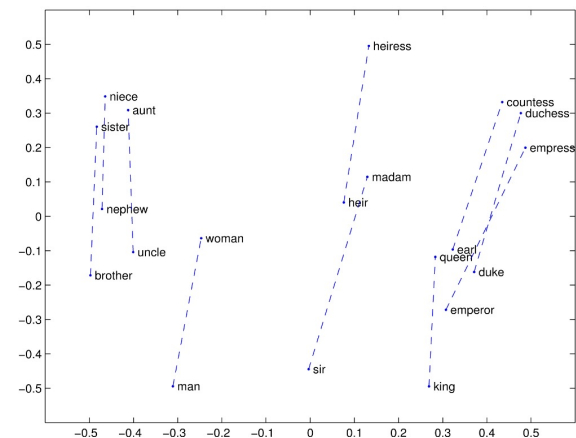
$$L = \sum_{i,j} f(X_{ij}) \quad (v_i \cdot u_j + b_i + \tilde{b}_j - \log X_{ij})^2$$

~~weighting squared reconstruction error~~

- Uses co-occurrence counts (the count-based tradition)
- Learns vectors by prediction (like word2vec)
- Faster to train: operates on the small matrix, not raw text

Vectors Encode Relationships, Not Just Similarity

- Analogical structure: relationships show up as consistent vector differences.
 - $\overrightarrow{\text{king}} - \overrightarrow{\text{man}} + \overrightarrow{\text{woman}} \approx \overrightarrow{\text{queen}}$
 - $\overrightarrow{\text{Paris}} - \overrightarrow{\text{France}} + \overrightarrow{\text{Italy}} \approx \overrightarrow{\text{Rome}}$
 - $\overrightarrow{\text{walked}} - \overrightarrow{\text{walk}} + \overrightarrow{\text{run}} \approx \overrightarrow{\text{ran}}$
- The model was never told what these relationships are. They emerged from text statistics.



Limitation: One Vector Per Word

- Static embeddings give every word exactly one vector.

*I deposited the check at the **bank**. ↔ We sat on the **bank** of the river.*

- Same word, two meanings. word2vec gives both the same vector: an awkward average.

That concert was sick.

- Positive (slang) or negative (literal)? Static embeddings can't tell. They have no access to context at use-time.
- The fix for polysemy: Make embeddings contextual. A word's vector should depend on the sentence it appears in.
- Also, we cannot use the models to train them directly on the task we want to solve.
- *That is the leap to BERT, GPT, and modern LLMs → next lecture*